

Trabajo Práctico 03

Planificación y trazabilidad

Alumno	Marcos Don
Matrícula	2319274
Repositorio	github.com/marcosdon28/IngeSoft3-2319274
Proyecto (público)	github.com/users/marcosdon28/projects/2
Sprint	Sprint 1 · 26/08 – 01/09/2026 · 1 semana
Límite de trabajo en progreso	2
Versión entregada	v3.0.0
Fecha	26/08/2026

1. Objetivo y qué se evalúa

Montar la gestión del proyecto sobre el mismo repositorio del semestre: jerarquía de trabajo, sprint, tablero, y **trazabilidad demostrable** entre lo que se pide y el código que lo implementa. El escenario del enunciado: el cliente pide visibilidad — quiere saber qué se está haciendo, qué falta, y cómo se conecta cada cambio con lo pedido.

Requisito	Estado
1 épica, 1 historia con criterios, 2 tareas, 1 bug	✓ issues #10 a #14
Jerarquía navegable (sub-issues, no task-lists)	✓ verificada por GraphQL
Sprint con duración elegida, con la historia y sus tareas	✓ Sprint 1, 1 semana
Tablero con automatización mínima (cerrar → Done)	✓ y se la vio actuar
Límite de trabajo en progreso	✓ 2 en <i>In Progress</i>
1 PR mergeado que cierra su issue automáticamente	✓ PR #16 → cierra #12
Proyecto público + <code>decisiones.md</code>	✓

Este práctico se lee distinto que el anterior. En el TP2 la guía era práctica y la entrega era otra cosa; acá **lo que la guía hace ES la entrega**. Y no lleva `evidencias.md`: el proyecto es público, así que quien corrige lo abre y lo ve en vivo. Las capturas de este informe son para estudiar, no para reemplazar esa URL.

2. La jerarquía: épica, historia, tareas — y el bug al costado

Tres niveles, y la diferencia entre ellos no es de tamaño sino de **qué responden**:

Nivel	Qué responde	¿Criterios de aceptación?
Épica #10	Cuál es el objetivo grande	No. No se verifica por sí misma: se cierra cuando sus historias están cerradas
Historia #11	Qué valor se entrega, y a quién	Sí. Es el nivel donde algo se puede comprobar
Tareas #12, #13	Qué hay que construir para entregarlo	No hace falta: se verifican contra la historia

marcosdon28 / IngeSoft3-2319274

Type to search

<> Code

Issues 4

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

EPIC: Pipeline DevOps completo para mi app #10

New issue

marcosdon28 opened 32m ago

Owner

Objetivo del semestre: construir el sistema de entrega completo de la app de inventario — CI, calidad automatizada, CD multi-entorno, contenedores en el pipeline, infraestructura como código y DevSecOps.

Esta épica **no lleva criterios de aceptación**: no se verifica por sí misma. Se da por cerrada cuando sus historias están cerradas.

Alcance

- CI: build y tests automáticos en cada PR (TP4, TP5)
- CD: entornos de QA y producción con aprobaciones (TP6)
- Release inmutable verificada end-to-end (TP7)
- Infraestructura como código (TP8)
- Seguridad automatizada y observabilidad (TP9)

Estado de partida

Ya resuelto en los prácticos anteriores:

- TP1 — flujo de trabajo con Pull Requests y `main` protegida
- TP2 — la app containerizada, con imágenes publicadas en ghcr

Sub-issues 0 of 1

> CI: build y tests automáticos en cada PR #11 1 of 2

Create sub-issue

marcosdon28 added epic 32m ago

marcosdon28 added a sub-issue 31m ago

CI: build y tests automáticos en cada PR #11

marcosdon28 added this to IngeSoft3 — Inventario DevOps 30m ago

github-project-automation moved this to Todo in IngeSoft3 — Inventario DevOps 30m ago

marcosdon28 moved this from Todo to In Progress in IngeSoft3 — Inventario DevOps 25m ago

marcosdon28 moved this from In Progress to Todo in IngeSoft3 — Inventario DevOps 25m ago

Add a comment

Write Preview

Use Markdown to format your comment

Paste, drop, or click to add files

Close issue

Comment

Assignees

No one — [Assign yourself](#)

Labels

epic

Projects

IngeSoft3 — Inventario DevOps

Status

Todo

Milestone

No milestone

Relationships

None yet

Development

Create a branch for this issue or link a pull request.

Notifications

Customize

Unsubscribe

You're receiving notifications because you're subscribed to this thread.

Participants

Open in GitHub Copilot app

Transfer issue

Clone issue

Lock conversation

Pin issue

Delete issue

Give feedback

© 2026 GitHub, Inc.

Terms

Privacy

Security

Status

Community

Docs

Contact

Manage cookies

Do not share my personal information

La épica con su historia colgando como **sub-issue** y la barra de progreso **0/1** : sigue abierta porque su historia sigue abierta.

Por qué sub-issues y no task-lists

Una task-list (`- [ ]` #12 escrito en el cuerpo) se ve casi igual y **no sirve para esto**: no crea la relación padre-hijo navegable, que es justamente lo que el enunciado pide. Con sub-issues se sube de la tarea a su historia y de ahí a la épica, y cada padre muestra una barra de progreso real.

El bug va al costado

La jerarquía cuenta **lo que se planificó construir**. Un bug es un defecto de algo **ya construido**: no era parte del plan, así que no forma parte del árbol. Colgarlo de la historia que lo originó tendría además un efecto feo — esa historia ya estaría cerrada, y su barra de progreso pasaría a mentir.

Matiz que conviene saber: hay equipos que sí los cuelgan, no para planificar sino para **medir** cuántos defectos se escapan por sprint; y en Azure Boards un Bug puede ser hijo de una Feature. O sea que «al costado» es una **convención de trabajo**, no una regla de la herramienta.

3. La historia y sus criterios de aceptación

marcosdon28 / IngeSoft3-2319274

Q Type to search

<> Code Issues 4 Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

CI: build y tests automáticos en cada PR #11

Open 1 / 2 Parent: EPIC: Pipeline DevOps completo para mi app

marcosdon28 opened 32m ago

Owner

Como desarrollador quiero que cada PR ejecute build y tests para detectar regresiones antes del merge.

Criterios de aceptación

- ☐ El workflow corre en cada PR a main
- ☐ Un test que falla bloquea el merge
- ☐ El reporte de tests queda publicado como artefacto
- ☐ Badge de estado visible en el README

Por qué cada criterio es verificable

Cada uno se comprueba mirando algo concreto, no opinando:

Criterio	Cómo se verifica
El workflow corre en cada PR a main	Se abre un PR y en la pestaña Checks aparece la corrida
Un test que falla bloquea el merge	Se rompe algo a propósito: el check queda en rojo y el botón de merge, deshabilitado
El reporte queda como artefacto	Al final de la corrida, en Artifacts, hay un archivo descargable
Badge visible en el README	Se abre el README y el badge está ahí, mostrando el estado del último build de main

Contraejemplo de criterio que no sirve: «que el CI funcione bien». No se puede comprobar — no dice qué mirar ni cuándo está cumplido.

Sub-issues 1 of 2

Escribir el workflow de build y tests #12

Publicar el reporte de tests como artefacto #13

Create sub-issue

marcosdon28 added story 32m ago

marcosdon28 mentioned this in 2 issues 32m ago

Escribir el workflow de build y tests #12

Publicar el reporte de tests como artefacto #13

marcosdon28 added a parent issue 31m ago

EPIC: Pipeline DevOps completo para mi app #10

marcosdon28 added sub-issues 31m ago

Escribir el workflow de build y tests #12

Publicar el reporte de tests como artefacto #13

marcosdon28 added this to IngeSoft3 — Inventario DevOps 30m ago

github-project-automation moved this to Todo in IngeSoft3 — Inventario DevOps 30m ago

marcosdon28 mentioned this 12m ago

ci: esqueleto del workflow de build y tests #16

Add a comment

Write Preview

Use Markdown to format your comment

Paste, drop, or click to add files

Close issue Comment

Assignees

No one - Assign yourself

Labels

story

Projects

IngeSoft3 — Inventario DevOps

Status

Todo

Milestone

No milestone

Relationships

Parent issue

EPIC: Pipeline DevOps completo para mi app

marcosdon28/IngeSoft3-2319274#10

Development

Create a branch for this issue or link a pull request.

Notifications

Unsubscribe

You're receiving notifications because you're subscribed to this thread.

Participants

Open in GitHub Copilot app

Transfer issue

Clone issue

Lock conversation

Pin issue

Delete issue

Give feedback

@ 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

La historia en formato *Como... quiero... para...*, con sus cuatro criterios y sus dos tareas como sub-issues. La barra marca 1/2 .

Qué hace verificable a un criterio

Un criterio sirve si dice **qué hay que mirar** para saber si está cumplido. Por eso el issue lleva, además de la lista, una tabla que lo explicita:

Criterio	Cómo se verifica
El workflow corre en cada PR a main	Se abre un PR y en <i>Checks</i> aparece la corrida
Un test que falla bloquea el merge	Se rompe algo a propósito: el check queda en rojo y el botón de merge, deshabilitado
El reporte queda como artefacto	Al final de la corrida, en <i>Artifacts</i> , hay un archivo descargable
Badge visible en el README	Se abre el README y el badge está ahí

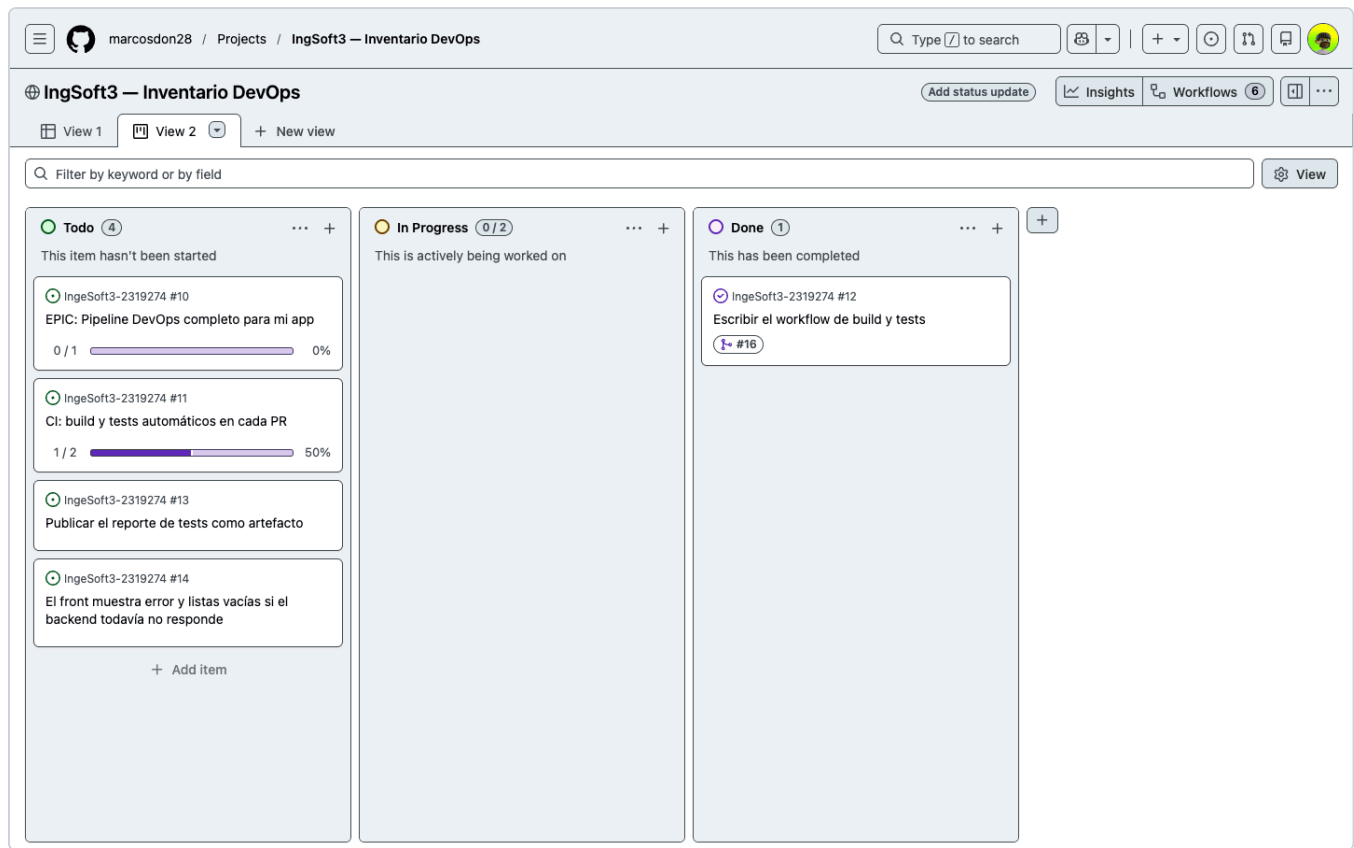
El contraejemplo que conviene tener a mano: «*que el CI funcione bien*» **no** es un criterio. No dice qué mirar ni cuándo está cumplido — dos personas pueden discutir para siempre si se cumple.

La historia mal escrita (el ejercicio)

Creé a propósito el issue #15 —«*Como desarrollador quiero crear la tabla usuarios para guardar los datos*»— para diagnosticarlo. Tiene la **forma** de historia y es una **tarea técnica disfrazada**: «para guardar los datos» no es un beneficio, es la definición de lo que hace una tabla; el «como desarrollador» tampoco ayuda, porque es quien la implementa y no quien recibe el valor; y no tiene nada verificable. Es además una historia **horizontal**: entrega una capa técnica, no una rebanada que alguien pueda usar.

Reescrita, con beneficiario y criterios: «*Como encargado de depósito quiero registrarme con usuario y contraseña para que cada movimiento de stock quede asociado a quien lo hizo*». La tabla `usuarios` sigue haciendo falta — pero pasa a ser una **tarea** de esa historia, que es su lugar. La diferencia no es de redacción: es de **nivel**.

4. El sprint y el tablero



El tablero: *Todo* con la épica, la historia (1/2), la tarea pendiente y el bug; *In Progress* con el límite **0 / 2**; *Done* con la tarea #12 y el chip del PR **#16** que la cerró.

Por qué el sprint dura una semana

Porque es el **ciclo real de entrega**: la cátedra publica un TP por semana y la defensa P1 cae en la clase 5. Con sprints de dos semanas el tablero mostraría un ritmo que no es el que tengo — planificaría contra un calendario y trabajaría contra otro.

El argumento general: el sprint debería durar lo que dura el **ciclo de feedback**. Más largo, y uno se entera tarde de que estimó mal; muy corto, y se gasta más tiempo en la ceremonia que en el trabajo.

Por qué el límite de trabajo en progreso es 2

La regla de arranque: **personas + 1**. Trabajando solo, 2. El «más uno» es la **válvula** para cuando algo queda esperando por fuera de mí —una revisión, un pipeline corriendo— y necesito avanzar sin dejar lo anterior a medias.

Por qué un límite en primer lugar: **el trabajo empezado y no terminado no vale nada**. Cuatro cosas al 60 % entregan cero; dos al 100 % entregan dos. El límite hace *visible* ese costo. Y la mecánica es justamente ésa: **no te lo impide, te lo muestra** — el contador de la columna se pone en rojo al pasarse.

- **Qué me haría subirlo**: sumar gente (la regla escala con personas), o descubrir que me bloqueo seguido esperando algo externo.

- **La señal de que quedó alto: si nunca lo alcanzo.** Un límite que nunca frena nada no limita: es decorativo. En diez, con una persona, sería el mismo tablero sin límite y con un número al lado.

```
marco — IngeSoft3-2319274 — sprint y estado del tablero

marco@MacBook IngeSoft3-2319274 % gh api graphql -f query='{ ... Sprint ... }'
campo: Sprint duracion por defecto: 7 dias
Sprint 1: inicio 2026-08-26, 7 dias

marco@MacBook IngeSoft3-2319274 % gh api graphql -f query='{ ... items ... }'
#10 sprint=- status=Todo EPIC: Pipeline DevOps completo para mi a
#11 sprint=Sprint 1 status=Todo CI: build y tests automáticos en cada PR
#12 sprint=Sprint 1 status=Done Escribir el workflow de build y tests
#13 sprint=Sprint 1 status=Todo Publicar el reporte de tests como artefa
#14 sprint=- status=Todo El front muestra error y listas vacías s

# La historia #11 y sus dos tareas #12 y #13 estan en Sprint 1.
# La epica #10 y el bug #14 NO llevan sprint: la epica abarca todo el
# semestre, y el bug todavia no fue priorizado en ninguna iteracion.
#
# status=Done en #12 lo puso SOLO el workflow 'Item closed -> Done'
# cuando se mergeo el PR #16. Nadie movio esa tarjeta a mano.
```

El sprint y el estado del tablero consultados por API: la historia y sus dos tareas en Sprint 1; la épica y el bug sin sprint. La épica abarca todo el semestre y el bug todavía no fue priorizado en ninguna iteración.

5. Trazabilidad: del requerimiento al código

La vuelta completa que pide el enunciado: desde una tarea cerrada llegar al PR y a los commits que la implementaron, y de ahí subir a su historia y a la épica.

Escribir el workflow de build y tests #12 New issue

Closed | Parent: CI: build y tests automáticos en cada PR | #16

marcosdon28 opened 32m ago Owner

Tarea técnica de la historia #11.

Crear `.github/workflows/ci.yml` con el disparador y el checkout del código: el esqueleto sobre el que el TP4 va a montar los jobs de build del backend y del frontend.

Qué incluye

- El archivo en `.github/workflows/`, que es donde GitHub busca sus workflows
- Disparador en `pull_request`
- Un job que corre en `ubuntu-latest` y hace `actions/checkout`

Fuera de alcance

Los builds reales de las imágenes y el cache de capas llegan en el TP4; los tests y su reporte, en el TP5. Hoy el workflow sólo tiene que existir y correr.

Create sub-issue

Timeline:

- marcosdon28 added task 32m ago
- marcosdon28 added a parent issue 31m ago
 - CI: build y tests automáticos en cada PR #11
- github-project-automation added this to IngSoft3 — Inventario DevOps 30m ago
- github-project-automation moved this to Todo in IngSoft3 — Inventario DevOps 30m ago
- marcosdon28 mentioned this 12m ago
 - ci: esqueleto del workflow de build y tests #16
- github-project-automation moved this from Todo to In Progress in IngSoft3 — Inventario DevOps 12m ago
- marcosdon28 closed this as completed in #16 12m ago
- github-project-automation moved this from In Progress to Done in IngSoft3 — Inventario DevOps 12m ago

Add a comment

Write | Preview

Use Markdown to format your comment

Paste, drop, or click to add files

Reopen issue | Comment

Assignees: No one - [Assign yourself](#)

Labels: task

Projects: IngSoft3 — Inventario DevOps (Status: Done)

Milestone: No milestone

Relationships: Parent issue: CI: build y tests automáticos en cada PR (marcosdon28/IngeSoft3-2319274#11) 1/2

Development: ci: esqueleto del workflow de build y tests (marcosdon28/IngeSoft3-2319274)

Notifications: Unsubscribe

Participants:

Open in GitHub Copilot app

Transfer issue

Clone issue

Lock conversation

Pin issue

Delete issue

Give feedback

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

La tarea #12 cerrada. Arriba: el chip **Parent** hacia la historia y el chip #16 del PR. En la línea de tiempo: `closed this as completed in #16` y `moved this from In Progress to Done`, hecho por `github-project-automation`. A la derecha: Status **Done**, el *Parent issue* con `1/2`, y la rama en *Development*.

Dos precisiones que cambian el resultado

Detalle	Por qué importa
Closes #12 va en la descripción del PR , no en el mensaje del commit	Por mensaje de commit el issue igual se cierra , pero no queda enlazado al PR — y ese enlace es lo que permite navegar de la tarea al código, y lo primero que se mira al corregir
El número es el de la tarea , no el de la historia	Un PR implementa una tarea concreta. Cerrando la historia #11 la estaría dando por terminada con la mitad del trabajo sin hacer: la tarea #13 sigue abierta. La trazabilidad quedaría mintiendo

El PR hace lo que el issue dice

La tarea #12 era «escribir el workflow de build y tests», y el PR crea exactamente ese archivo:

```
# .github/workflows/ci.yml
name: CI
on: [pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

Hoy el workflow sólo tiene que **existir y correr**: los builds reales de las dos imágenes y el cache llegan en el TP4 —que va a reemplazar el job `build` por `build-backend` y `build-frontend` — y los tests con su reporte, en el TP5. Esa parte es la tarea #13, que **queda abierta a propósito**.

Un PR que cierra un issue que no implementó es trazabilidad de mentira: el issue queda cerrado y el trabajo, sin hacer. Es lo primero que se mira al corregir.

El estado en el que queda la entrega

Issue	Estado	Por qué
#12 — tarea del workflow	Cerrada	La cerró el PR #16
#13 — tarea del artefacto	Abierta	Depende de los tests, que llegan en el TP5
#11 — la historia	Abierta (1/2)	Falta una de sus dos tareas
#10 — la épica	Abierta (0/1)	Su historia sigue abierta
#14 — el bug	Abierta	Al costado, sin sprint: no fue priorizado todavía

Es el estado correcto, y conviene poder explicarlo: la automatización mueve tarjetas, pero **no cierra una historia porque se hayan cerrado sus sub-issues**. La barra puede llegar a 2/2 y la historia sigue abierta hasta que alguien la cierre.

6. El bug: uno propio, no el de la guía

El enunciado permite usar el bug del video, pero dice que uno propio es mejor. Usé uno **real de mi app**, que puedo reproducir en vivo.

Qué pasa: al levantar el sistema con `docker compose up -d` y abrir la app enseguida, se ve el cartel de error con las listas vacías — y **no se recupera solo** aunque el backend termine de arrancar dos segundos después.

Por qué: en `frontend/src/App.jsx`, `recargar()` está memoizada con `useCallback(..., [])` y se dispara desde un `useEffect` que corre **una sola vez**. El `catch` guarda el mensaje y ahí termina: no hay reintento ni estado de carga.

```
const recargar = useCallback(async () => {
  try {
    const [p, c, m] = await Promise.all([...])
    setProductos(p); setCategorias(c); setMovimientos(m); setError(null)
  } catch (e) {
    setError(e.message) // ← se muestra el error y no se vuelve a intentar
  }
}, [])

useEffect(() => { recargar() }, [recargar]) // ← corre una sola vez
```

El `healthcheck` del compose hace que **el backend** espere a que la base esté lista, pero nginx sirve la SPA apenas arranca: no hay nada que retenga al browser hasta que la API responda.

Por qué es un bug y no «trabajo que faltaba»: aparece sobre algo **ya entregado** — la app del TP2, que está cerrada y etiquetada `v2.0.0`. Si lo hubiera encontrado *mientras* esa historia estaba en curso, no sería un bug: sería que la historia todavía no cumplía sus criterios de aceptación. El principio que ordena las dos situaciones es uno solo: **una historia con defectos no está terminada**.

7. Problemas encontrados y aprendizajes

El CLI no puede crear campos *Iteration*, pero la API sí

`gh project field-create` sólo acepta `TEXT`, `NUMBER`, `DATE` y `SINGLE_SELECT`, así que di por sentado que el campo Sprint había que crearlo a mano. Probé igual la mutación `createProjectV2Field` con `dataType: ITERATION` y **funcionó**.

La lección excede a la herramienta: que una interfaz no exponga algo no significa que la plataforma no lo soporte. Vale la pena mirar el esquema antes de resignarse.

...pero el campo nace sin configurar, y eso sí es sólo web

La mutación crea el campo con `duration: 0` y cero iteraciones: no existe mutación pública para definir la duración ni generar las iteraciones. Una vez configurado desde la interfaz, en cambio, **asignar el sprint a cada item sí se puede por API**, con `updateProjectV2ItemFieldValue` y el `iterationId`.

Dónde frené de automatizar, y por qué

Intenté configurar la iteración, crear la vista de tablero y poner el límite con Playwright, y no fue viable: Projects v2 es una aplicación propia donde los controles no exponen roles accesibles estándar. El botón *New view* no es un `button` con nombre accesible; el selector de fechas usa `div`s con `aria-label` dinámicos del tipo `"Tuesday, September 1 (Inside selected range)"`, que cambian según el estado de la selección.

Después de varios intentos frené y lo hice a mano. **Automatizar algo que tarda treinta segundos por interfaz no se justifica**, y forzar selectores frágiles hubiera dejado un procedimiento que se rompe con el próximo rediseño.

Una verificación apurada me hizo dar por fallado algo que funcionó

Consulté el campo Sprint inmediatamente después de guardarlo y la API devolvió cero iteraciones, así que lo di por fallado. Al rato el mismo query devolvía la iteración de 7 días correctamente guardada: GitHub propaga el cambio de forma asíncrona. **Verificar demasiado rápido puede ser peor que no verificar**, porque uno actúa sobre una conclusión falsa — en este caso, casi rehago a mano algo que ya estaba hecho.

Los Projects de usuario nacen privados

Y el entregable de este práctico **es la URL**: entregada privada, quien la abre ve un 404 — ni siquiera «no tenés permiso», sino «no existe». Se cambia con `gh project edit 2 --owner "@me" --visibility PUBLIC`, que además es **idempotente**. El control antes de entregar: abrir la propia URL en una ventana de incógnito.

Declaración de uso de Inteligencia Artificial

Qué fue asistido	Cómo lo verifiqué
Labels, issues, jerarquía y Project por CLI	Consulté la jerarquía de vuelta por GraphQL (subIssues) en vez de mirar la pantalla: épica → historia → dos tareas, con el bug fuera del árbol
La historia y sus criterios de aceptación	Revisé cada criterio preguntándome cómo se comprueba . El que no pasaba esa prueba no entraba — por eso el issue lleva una tabla que dice, para cada uno, qué hay que mirar
El PR de trazabilidad	Verifiqué que el PR hace lo que el issue dice , y después por API que #12 quedó cerrada y enlazada al PR #16, y que #11, #13 y #10 siguen abiertas
El bug	Lo encontré leyendo mi propio código y lo reproduje: <code>down -v && up -d</code> y abrir la app enseguida. El issue cita la línea exacta

Lo que **no** delegué: la duración del sprint, el límite de trabajo en progreso, el diagnóstico de la historia mal escrita, y la decisión de usar un bug propio en vez del de la guía.

Anexo — Los comandos, en orden

```
# — Permiso que faltaba —————
gh auth refresh -h github.com -s project

# — Etiquetas (bug ya viene de fábrica) —————
gh label create epic --color 6f42c1 --description "Épica"
gh label create story --color 0e8a16 --description "Historia de usuario"
gh label create task --color 1d76db --description "Tarea técnica"

# — La jerarquía —————
gh issue create --title 'EPIC: Pipeline DevOps completo para mi app' --label epic --body '...'
gh issue create --title 'CI: build y tests automáticos en cada PR' --label story --body '...'
gh issue create --title 'Escribir el workflow de build y tests' --label task --body '...'
gh issue create --title 'Publicar el reporte de tests como artefacto' --label task --body '...'
gh issue create --title 'El front muestra error si el backend no responde' --label bug --body '...'

gh issue edit 10 --add-sub-issue 11      # épica → historia
gh issue edit 11 --add-sub-issue 12      # historia → tarea
gh issue edit 11 --add-sub-issue 13

# — El proyecto —————
gh project create --owner "@me" --title "IngSoft3 – Inventario DevOps"
gh project edit 2 --owner "@me" --visibility PUBLIC      # nacen PRIVADOS
for n in 10 11 12 13 14; do gh project item-add 2 --owner "@me" --url ../issues/$n; done

# — El campo Sprint: el CLI no puede, la API sí —————
gh api graphql -f query='mutation { createProjectV2Field(input: {
  projectId: "PVT_...", dataType: ITERATION, name: "Sprint" }) { ... } }'
# ... la duración y las iteraciones se configuran desde la interfaz ...

# — Asignar el sprint (esto SÍ por API) —————
gh api graphql -f query='mutation { updateProjectV2ItemFieldValue(input: {
  projectId: "PVT_...", itemId: "PVTI_...", fieldId: "PVTIF_...",
  value: { iterationId: "91d15287" } }) { ... } }'

# — Trazabilidad —————
git switch -c ci/workflow-de-build-y-tests
mkdir -p .github/workflows      # ... escribir ci.yml ...
git add .github/workflows/ci.yml && git commit -m 'ci: esqueleto del workflow'
git push -u origin ci/workflow-de-build-y-tests
gh pr create --title '...' --body '... Closes #12'      # ← en la DESCRIPCIÓN
gh pr merge 16 --squash --delete-branch

# — Verificación —————
gh issue view 12 --json state      # CLOSED
gh issue view 11 --json state      # OPEN – la historia sigue abierta
gh api graphql -f query='{ ... subIssuesSummary ... }'      # 1/2 (50%)

# — Cerrar el práctico —————
git tag -a v3.0.0 -m "TP3 cerrado" && git push origin v3.0.0
```